

Project Phase 1 Report

Betül Merey 33963

Nehir Ceylan 35242

Project Title: Consultancy Appointment and Management System (CAMS)

Project Description

The Appointment and Service Management System is a specialized digital framework designed to orchestrate the interactions between professional service providers and their clientele. At the center of the architecture lies the Appointment entity, which serves as the central hub connecting all other system components. Each session is uniquely distinguished by a primary identification code and a specific temporal marker to ensure precise scheduling and record-keeping.

A Client initiates the process by maintaining a profile that includes unique personal identifiers, birth details for age-sensitive services, and essential communication channels like email and telephone. While a single client can manage and book multiple sessions over time, each specific session is tied back to one primary recipient. Similarly, a Professional fulfills these requests by providing their specific area of expertise, legally required license credentials, and total duration of industry experience. The system is designed so that one professional can conduct numerous appointments, effectively managing their entire service portfolio within the database.

To ensure versatility, every session is associated with a specific Service definition, categorized by its name and a detailed explanation of what the task entails. Furthermore, each interaction is assigned to a Location; however, this is not restricted to physical infrastructure. The system tracks these venues through their specific addresses and environment types, accommodating both traditional facility based meetings and modern virtual/online settings.

The lifecycle of a session is completed through integrated financial and quality-control modules. A Payment is generated for each appointment, acting as a dependent record that tracks the monetary amount, the date of the transaction, and the specific status or method used for the transfer. Finally, to maintain service standards, a Review can be linked to a concluded session. This allows the client to provide a quantitative rating and descriptive qualitative feedback.

Entities and Attributes

Appointment : The central hub of the system that organizes the scheduled meetings.

- Attributes: Appointment_id (PK), date

Client : Individuals who receive the service.

- Attributes: phone, email, date_of_birth, client_id (PK), name

Professional: the service provider registered in the database

- Attributes: name, professional_id (PK), years_of_experience, license_number, specialization

Service: categorizes the various professional offerings available.

- Attributes: service_id (PK), description, service_name

Location : the space where the service is provided.

- Attributes: address, location_id(PK), location_type

Payment: A weak entity that handles the financial details of an appointment.

- Attributes: payment_id (Partial Key), payment_method, amount, payment_status, payment_date

Review: A weak entity designed to capture user feedback after a session.

- Attributes: review_id (Partial Key), comment, rating

Relationships

BOOKS (Client to Appointment) : An one to many relationship. Each appointment is booked by exactly one client, while a client is permitted to book multiple appointments over time.

CONDUCTS (Professional to Appointment): An one to many relationship. A professional is responsible for conducting various appointments, but each specific appointment is tied to only one professional.

TAKES PLACE AT (Appointment to Location): A many to one relationship. Multiple appointments can be scheduled at the same location, whereas a single appointment must occur at exactly one designated venue.

FOR (Appointment to Service): A many to one relationship. Every appointment is initiated for one specific type of service, although a single service category can be associated with many different appointments.

HAS (Appointment to Payment): An one to one identifying relationship. Each appointment generates exactly one payment record. As a weak entity, the payment is uniquely identified in the context of its parent appointment.

HAS (Appointment to Review): An one to one identifying relationship. An appointment can result in one review session. This is a weak relationship where the review depends on the existence of a completed appointment.

Participation and Key Constraints

Every **Appointment** must have a corresponding **Client**, **Professional**, **Service**, and **Location** to be valid. This is represented by the thick lines connecting Appointment to these entities.

Every **Payment** must be linked to an **Appointment**, as a transaction cannot exist without a scheduled service.

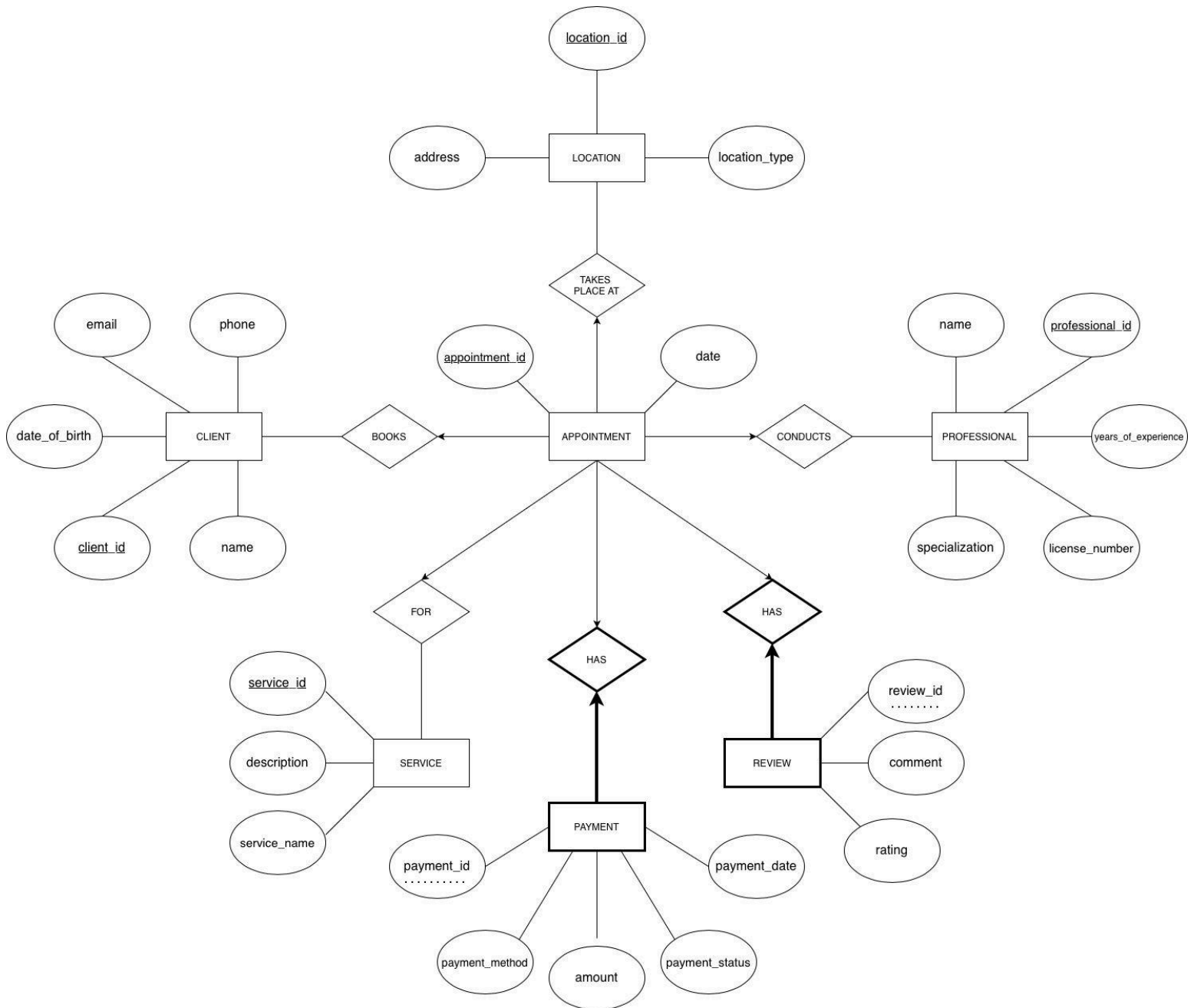
Every **Review** must be associated with a specific **Appointment**.

Professionals, **Clients**, and **Locations** may exist in the system without being currently linked to an active appointment

appointment_id, *client_id*, *professional_id*, *service_id*, and *location_id* serve as **Primary Keys** for their respective entities.

payment_id and *review_id* act as **Partial Keys**, requiring the appointment's primary key for unique identification.

ER DIAGRAM (MODEL)



Appendix (AI Tool: Gemini)

Prompt 1:

(Examples from the lecture were attached)

We are working on a project for database systems course, it is needed to design ER diagram. We thought concept for a Consultancy Appointment System, and I want to get your feedback on my initial design before finalizing the details. I added some course examples to show specific format. The system is designed to manage appointments between clients and professionals across various service locations. Structure-Entities&Attributes: Client (id , name, email, phone, date of birth) - Professional (id, name, specialization, license number, years of experience) - Service (id , address, city, room number) - appointment (id, date, start time) - payment (id , date, amount, method, status).

Relationships&constraints: Client-Books-Appointment(Each appointment must strictly belong to one client) / Professional-Conducts-Appointment(A professional is assigned to conduct specific appointments) / Appointment-Includes-Services(for each appointment - exactly one specific service) /

Appointment-Takes Place At-Location: Every appointment must be linked to a physical location (Service entity) / Appointment-Has-Payment: A payment must be tied to a specific appointment to be valid.

This is what I thought I need feedback. Should I include something else or are there any logical mistakes. I will add weak entities and relations like many to many or one to one or many to one or one to many.

AI Response:

It's great to see you applying the ER modeling concepts from your course to a practical system! Based on the images you provided (which cover ISA hierarchies, Weak Entities, and Aggregation), I've reviewed your design.

Your initial structure is a solid start, but there are a few logical "traps" and structural improvements you should consider before you start drawing.

1. The "Service" vs. "Location" Ambiguity

In your description, you've defined **Service** as having attributes like `address`, `city`, and `room number`.

- The Issue:** Logically, a "Service" is usually an action (e.g., "Tax Consultation" or "Legal Advice"), whereas an `address` and `room number` describe a **Location**.
- Recommendation:** Rename your current "Service" entity to **Location**. If you want to track what *kind* of service is being provided (e.g., Physiotherapy vs. Dietician), create a separate **Service** entity. An Appointment would then link to both a Service (what is being done) and a Location (where it is happening).

2. Identifying Weak Entities

Looking at your "Better Design" example (Image 2) involving **Dependents**, you should consider if any of your entities are "Weak."

- Payment as a Weak Entity:** A `Payment` usually cannot exist without an `Appointment`. If a payment doesn't have its own independent meaning outside of the appointment it pays for, you can model it as a **Weak Entity** or a **one-to-one relationship** with total participation from the Payment side.
- Location/Room:** If "Room Number" is only unique within a specific "Building," the Room becomes a Weak Entity of the Building.

3. Using ISA Hierarchies (Subtyping)

Referencing your first image (ISA triangle), think about whether your `Professional` or `Client` entities need subtypes.

3. Using ISA Hierarchies (Subtyping)

Referencing your first image (ISA triangle), think about whether your `Professional` or `Client` entities need subtypes.

- Example:** If you have different types of Professionals (e.g., `Medical_Staff` vs. `Admin_Staff`), you could use an ISA hierarchy. `Medical_Staff` would have `license_number`, while `Admin_Staff` might not. This prevents having "null" attributes in a single large table.

4. Relationship Cardinality & Participation

Since you mentioned you'll be adding these next, keep these "Business Rules" in mind:

- Client - Books - Appointment:** This is likely 1:N (One client can have many appointments, but one appointment belongs to one client).
- Total Participation:** Does every Appointment *must* have a Professional? Yes. This means a **thick line** (or double line) connecting `Appointment` to the `Conducts` relationship.

Suggested Refined ER Structure

Entity	Key Attributes	Potential Logic
Client	id, name, email	Strong Entity
Professional	id, name, license	Strong Entity
Location	id, city, address	Strong Entity
Appointment	id, date, time	The "Central" Entity
Payment	payment_num	Weak Entity (depends on Appointment)

5. Advanced Concept: Aggregation

5. Advanced Concept: Aggregation

Looking at your fourth image (the **Monitors** example with a dashed box), you might need **Aggregation** if you want to record something about a relationship.

- Example:** If you want to record a `Review` for an appointment, the Review isn't just for the Professional, it's for the specific *occurrence* of the Professional-Client-Appointment interaction. You would "aggregate" that relationship and link the `Review` entity to it.

Logical Checklist for your Next Draft:

- Multi-valued Attributes:** Does a Professional have multiple phone numbers? If so, use a double oval.
- Derived Attributes:** Do you have `years_of_experience`? You could derive this from a `start_date` (use a dashed oval).
- Many-to-Many:** Can an appointment include *multiple* services? If so, that relationship will eventually become its own table.

Directly Used:

- **Entity Separation:** Following the AI's recommendation regarding the "Service vs. Location" ambiguity, we explicitly separated the physical infrastructure from the action. We renamed the initial location-based entity to **Location** and created a new **Service** entity to categorize the specific type of professional assistance being provided.
- **Review Feature:** The concept of incorporating a feedback mechanism for appointments was adopted based on the AI's suggestion to record the quality of the interaction.

Modified:

- **Weak Entity Implementation:** Inspired by the AI's "Identifying Weak Entities" advice, we modeled both **Payment** and **Review** as weak entities. While the AI suggested a generic dependent structure, we specifically implemented these with **Partial Keys** (*payment_id* and *review_id*) and **Identifying Relationships** (double diamonds) to ensure they are functionally dependent on a specific **Appointment**.
- **Aggregation Logic:** The AI suggested using **Aggregation** to link the Review to the Professional-Client interaction. We modified this logic by directly linking the **Review** to the **Appointment** as a weak entity, which achieved the same goal of recording a specific occurrence while remaining consistent with our primary hub-based design.

Rejected:

- **ISA Hierarchies:** The suggestion to use ISA triangles (subtyping) for Professionals or Clients was rejected to maintain a streamlined schema and avoid null-attribute management complexity at this stage.
- **Multivalued/Derived Attributes:** Recommendations for multivalued phone numbers or derived *years_of_experience*(from a start date) were rejected. We chose to keep *years_of_experience* as a static attribute because the system requires the professional's total career experience rather than just their tenure within this specific platform.